

MACHINE LEARNING

Client Churn ML Prediction

Rapport académique – Analyse comportementale clientèle retail

Auteur / Étudiant :	Anis Kchaou
Professeur :	Mme Fadoua Drira
Groupe :	GI2 – S3
Repository :	github.com/Anis-Kchaou/Client_churn_ML_prediction
Langage :	Python 3
Frameworks :	Scikit-learn, Flask
Dataset :	4 372 clients, 52 features brutes
Année universitaire :	2025–2026

Table des matières

Résumé	3
1 Contexte et objectifs du projet	4
1.1 Contexte métier	4
1.2 Enjeux business	4
1.3 Objectifs pédagogiques	4
2 Configuration de l'environnement de travail	4
2.1 Structure générale du projet	5
2.2 Installation	5
3 Description du dataset	7
3.1 Vue d'ensemble	7
3.2 Catégories de variables	7
3.3 Distribution de la cible	7
4 Qualité des données et stratégie de nettoyage	9
4.1 Problèmes identifiés	9
4.2 Traitements appliqués	9
4.3 Cas des valeurs négatives	9
5 Pipeline de prétraitement	9
5.1 Étapes principales	9
5.2 Feature engineering	10
5.3 Encodage des variables	10
5.4 Normalisation et data leakage	11
6 Modélisation machine learning	12
6.1 Analyse en composantes principales	12
6.2 Random Forest Classifier	12
6.3 Régression logistique	12
6.4 KMeans	13

6.5	Random Forest Regressor	13
7	Déploiement avec Flask	14
7.1	Objectif du déploiement	14
7.2	Architecture de l'application	14
7.3	API de prédiction	14
7.4	Avantages de Flask	14
8	Analyse des résultats et discussion	15
8.1	Analyse descriptive	15
8.2	Points forts du projet	15
8.3	Limites identifiées	15
8.4	Améliorations proposées	16
9	Conformité avec les exigences de l'atelier	17
10	Conclusion	18

Vue globale

Ce rapport présente une étude académique d'un projet de machine learning appliqué à la prédiction du churn client dans un contexte e-commerce. Le projet vise à analyser le comportement des clients, prédire leur probabilité d'attrition, segmenter la clientèle et proposer une application web permettant l'utilisation du modèle en conditions opérationnelles.

Le dataset utilisé contient 4 372 clients décrits initialement par 52 variables brutes. Après nettoyage, transformation et encodage, le jeu de données est réduit à 49 variables exploitables. La variable cible, *Churn*, est binaire : elle indique si un client est fidèle ou churné. Le projet comprend un pipeline complet allant de l'exploration des données au déploiement via Flask.

Mots-clés : churn client, machine learning, e-commerce, Random Forest, ACP, KMeans, Flask, Scikit-learn, segmentation client.

1. Contexte et objectifs du projet

1.1. Contexte métier

Dans le secteur du commerce électronique, la fidélisation client constitue un enjeu stratégique majeur. Une entreprise spécialisée dans la vente de cadeaux souhaite exploiter ses données transactionnelles et comportementales afin de mieux comprendre ses clients et d'anticiper leur départ.

Le churn client correspond à la perte d'un client, c'est-à-dire au fait qu'il cesse d'acheter ou d'interagir avec l'entreprise. Dans un contexte concurrentiel, identifier les clients à risque permet de mettre en place des actions préventives : campagnes marketing ciblées, offres personnalisées, relances commerciales ou amélioration du service client.

1.2. Enjeux business

Les principaux enjeux du projet sont les suivants :

- **Réduire le churn** : identifier les clients susceptibles de quitter la plateforme.
- **Personnaliser le marketing** : segmenter les clients selon leurs comportements d'achat.
- **Optimiser le chiffre d'affaires** : concentrer les efforts sur les clients à forte valeur.
- **Industrialiser la prédiction** : déployer un modèle utilisable via une interface web.

1.3. Objectifs pédagogiques

Ce projet répond aux exigences d'un atelier de machine learning en GI2. Il couvre les principales étapes d'un cycle de vie ML :

1. exploration et analyse des données ;
2. nettoyage et préparation du dataset ;
3. feature engineering ;
4. encodage des variables ;
5. séparation train/test ;
6. entraînement de modèles supervisés et non supervisés ;
7. évaluation des résultats ;
8. déploiement via Flask.

2. Configuration de l'environnement de travail

2.1. Structure générale du projet

Le dépôt du projet suit une organisation modulaire, ce qui facilite la maintenance, la lisibilité et la réutilisation du code. La structure générale est la suivante :

```
Client_churn_ML_prediction/  
|-- data/  
|   |-- raw/  
|   |-- processed/  
|   |-- train_test/  
|  
|-- notebooks/  
|   |-- exploration.ipynb  
|  
|-- src/  
|   |-- preprocessing.py  
|   |-- train_model.py  
|   |-- train_classification.py  
|   |-- train_clustering.py  
|   |-- train_regression.py  
|   |-- pca_transform.py  
|   |-- predict.py  
|   |-- utils.py  
|  
|-- models/  
|  
|-- app/  
|   |-- app.py  
|  
|-- reports/  
|  
|-- requirements.txt  
|-- README.md
```

Cette organisation sépare clairement les données, le code source, les modèles entraînés, les notebooks d'exploration et l'application de déploiement.

2.2. Installation

Les commandes principales pour installer le projet sont :

```
git clone https://github.com/Anis-Kchaou/Client_churn_ML_prediction.git  
cd Client_churn_ML_prediction
```

```
python -m venv venv  
venv\Scripts\activate  
pip install -r requirements.txt
```

L'utilisation d'un environnement virtuel permet d'isoler les dépendances du projet et d'éviter les conflits entre bibliothèques Python.

3. Description du dataset

3.1. Vue d'ensemble

Le dataset contient des informations relatives aux clients d'une plateforme e-commerce. Il est composé de données transactionnelles, comportementales, démographiques et techniques.

2lightgraywhite	
Élément	Description
Nombre de clients	4 372
Nombre de features brutes	52
Nombre de features après prétraitement	49
Variable cible	Churn
Type de cible	Binaire : 0 = fidèle, 1 = churné
Distribution	66,7% clients fidèles / 33,3% clients churnés
Split	80% apprentissage / 20% test

TABLE 1 – Vue d'ensemble du dataset

3.2. Catégories de variables

Les variables peuvent être regroupées en plusieurs catégories :

- **Variables RFM** : récence, fréquence, montant total dépensé.
- **Variables transactionnelles** : quantité totale, nombre de commandes, nombre de produits uniques.
- **Variables comportementales** : ratio d'achats en weekend, délai moyen entre commandes.
- **Variables démographiques** : âge, genre, région, pays.
- **Variables d'engagement** : satisfaction, tickets support, niveau de fidélité.
- **Variables techniques** : date d'inscription, adresse IP de dernière connexion.

3.3. Distribution de la cible

La variable *Churn* présente un déséquilibre modéré. Les clients fidèles représentent environ deux tiers de la population, tandis que les clients churnés représentent environ un tiers.

Classe	Libellé	Effectif	Proportion
0	Client fidèle	2 918	66,7%
1	Client churné	1 454	33,3%
Total	–	4 372	100%

TABLE 2 – Distribution de la variable cible Churn

Le déséquilibre des classes doit être pris en compte lors de l'évaluation du modèle. Des métriques comme le rappel, le F1-score ou l'AUC sont plus pertinentes qu'une simple accuracy.

4. Qualité des données et stratégie de nettoyage

4.1. Problèmes identifiés

Le dataset présente plusieurs problèmes de qualité, volontairement intégrés pour simuler un cas réel :

- valeurs manquantes dans certaines variables comme l'âge ;
- valeurs aberrantes dans *SupportTickets* et *Satisfaction* ;
- formats de dates inconsistants ;
- variable constante *Newsletter* ;
- valeurs négatives dans certaines variables monétaires ou quantitatives ;
- forte dispersion de certaines variables transactionnelles.

4.2. Traitements appliqués

Problème	Variables concernées	Traitement
Valeurs manquantes	Age, AvgDaysBetween	Imputation KNN avec 5 voisins
Valeurs aberrantes	SupportTickets	Remplacement de -1 et 999 par NaN
Valeurs aberrantes	Satisfaction	Remplacement de -1, 0 et 99 par NaN
Date inconsistante	RegistrationDate	Conversion avec <code>pd.to_datetime</code>
Variable constante	Newsletter	Suppression
Adresse IP brute	LastLoginIP	Extraction d'une variable binaire IP_Prive

TABLE 3 – Synthèse des problèmes de qualité et traitements

4.3. Cas des valeurs négatives

Certaines variables comme *MonetaryTotal* ou les quantités peuvent contenir des valeurs négatives. Dans un contexte e-commerce, ces valeurs peuvent correspondre à des retours, remboursements ou annulations. Elles ne sont donc pas automatiquement supprimées, car elles peuvent contenir une information importante sur le comportement client.

5. Pipeline de prétraitement

5.1. Étapes principales

Le script `preprocessing.py` met en œuvre un pipeline complet en plusieurs étapes :

1. chargement du fichier brut `clients.csv` ;
2. suppression des variables inutiles ;
3. parsing de la date d'inscription ;
4. création de nouvelles variables ;
5. correction des valeurs aberrantes ;
6. encodage ordinal des variables catégorielles ordonnées ;
7. encodage one-hot des variables nominales ;
8. target encoding de la variable *Country* ;
9. séparation des variables explicatives et de la cible ;
10. imputation des valeurs manquantes ;
11. séparation train/test stratifiée ;
12. normalisation des données.

5.2. Feature engineering

Le feature engineering permet d'enrichir le dataset avec des variables plus expressives. Les variables créées sont les suivantes :

Variable	Formule	Interprétation
MonetaryPerDay	$\text{MonetaryTotal} / (\text{Recency} + 1)$	Dépense moyenne par jour récent
AvgBasketValue	$\text{MonetaryTotal} / \text{Frequency}$	Valeur moyenne du panier
TenureRatio	$\text{Recency} / \text{CustomerTenure}$	Rapport entre inactivité et ancienneté
RegYear	Année de RegistrationDate	Effet temporel annuel
RegMonth	Mois de RegistrationDate	Saisonnalité de l'inscription
IP_Prive	Détection IP privée	Indicateur technique de connexion

TABLE 4 – Variables créées par feature engineering

5.3. Encodage des variables

Les variables catégorielles sont traitées selon leur nature :

- les variables ordinales sont transformées avec un encodage ordinal ;
- les variables nominales sont converties par one-hot encoding ;
- la variable *Country*, ayant de nombreuses modalités, est traitée par target encoding.

5.4. Normalisation et data leakage

La normalisation est réalisée avec **StandardScaler**. Le scaler doit être ajusté uniquement sur l'ensemble d'apprentissage, puis appliqué à l'ensemble de test. Cette précaution évite le *data leakage*, c'est-à-dire l'utilisation indirecte d'informations provenant du test pendant l'apprentissage.

Une bonne pratique consiste à ajuster tous les transformateurs statistiques, comme l'imputer ou le scaler, uniquement sur **X_train**, puis à transformer **X_test**.

6. Modélisation machine learning

6.1. Analyse en composantes principales

L'Analyse en Composantes Principales, ou ACP, est utilisée pour réduire la dimensionnalité des données. Dans ce projet, les variables normalisées sont projetées dans un espace de 10 composantes principales.

L'ACP est particulièrement utile pour :

- réduire le bruit ;
- faciliter le clustering ;
- limiter la multicolinéarité ;
- visualiser les données dans un espace réduit.

6.2. Random Forest Classifier

Le modèle principal de classification est le *Random Forest Classifier*. Il est adapté à ce type de problème car il gère bien les relations non linéaires, les interactions entre variables et les variables issues d'un encodage multiple.

Le Random Forest est utilisé pour prédire la probabilité qu'un client appartienne à la classe churnée.

Les métriques utilisées sont :

- précision ;
- rappel ;
- F1-score ;
- ROC-AUC.

6.3. Régression logistique

La régression logistique est utilisée comme modèle de référence. Elle est entraînée sur les composantes issues de l'ACP. Ce choix permet de réduire les effets de multicolinéarité et d'obtenir un modèle plus simple à interpréter.

6.4. KMeans

Le modèle KMeans est utilisé pour segmenter les clients en quatre groupes homogènes. Cette segmentation permet d'identifier différents profils de clients, par exemple :

- clients champions ;
- clients fidèles ;
- clients à risque ;
- clients dormants.

6.5. Random Forest Regressor

Un module de régression est également présent. Il peut être utilisé pour prédire une valeur continue, comme une valeur client future ou un score de propension.

Modèle	Type	Objectif
Random Forest Classifier	Classification supervisée	Prédire le churn
Logistic Regression	Classification supervisée	Modèle baseline sur ACP
KMeans	Clustering non supervisé	Segmenter les clients
Random Forest Regressor	Régression supervisée	Prédire une valeur continue
ACP	Réduction dimensionnelle	Réduire les 47 variables à 10 composantes

TABLE 5 – Comparatif des modèles implémentés

7. Déploiement avec Flask

7.1. Objectif du déploiement

Le déploiement vise à rendre le modèle utilisable par un utilisateur final. Au lieu d'exécuter manuellement des scripts Python, l'utilisateur peut saisir les informations d'un client dans une interface web ou envoyer une requête API.

7.2. Architecture de l'application

L'application est développée avec Flask. Elle charge les objets sauvegardés :

- l'imputer ;
- le scaler ;
- le modèle Random Forest.

Le flux de prédiction est le suivant :

**Données client → Imputation KNN → Standardisation → Random Forest →
Prédiction churn**

7.3. API de prédiction

Le module `predict.py` expose une fonction de prédiction qui reçoit un dictionnaire de données client et retourne une sortie structurée, par exemple :

```
{  
    "churn": 1,  
    "probabilite_churn": 0.847  
}
```

7.4. Avantages de Flask

Flask est adapté à ce projet pour plusieurs raisons :

- simplicité de prise en main ;
- compatibilité directe avec les modèles Scikit-learn ;
- possibilité de créer rapidement une API REST ;
- facilité d'intégration avec une interface HTML/CSS ;
- bon choix pédagogique pour découvrir le déploiement ML.

8. Analyse des résultats et discussion

8.1. Analyse descriptive

Les statistiques descriptives montrent une forte hétérogénéité entre les clients. Certaines variables présentent une dispersion importante, notamment les montants dépensés et la fréquence d'achat. Cela indique l'existence de profils très différents : clients occasionnels, clients réguliers, clients très actifs ou clients professionnels.

Les valeurs extrêmes observées dans les montants ou les quantités ne doivent pas être supprimées systématiquement. Elles peuvent refléter des comportements réels, comme des achats en gros ou des retours de produits.

8.2. Points forts du projet

Le projet présente plusieurs points forts :

- pipeline de prétraitement complet ;
- gestion des valeurs manquantes et aberrantes ;
- feature engineering pertinent basé sur les métriques RFM ;
- combinaison de classification, clustering et régression ;
- sauvegarde des modèles avec `joblib` ;
- application Flask pour le déploiement ;
- structure de projet claire et modulaire.

8.3. Limites identifiées

Malgré sa qualité, le projet présente certaines limites :

- le déséquilibre des classes peut réduire la performance sur la classe churnée ;
- l'imputation KNN devrait idéalement être ajustée uniquement sur l'ensemble d'apprentissage ;
- les hyperparamètres ne sont pas encore optimisés ;
- la validation croisée n'est pas suffisamment exploitée ;
- le monitoring en production n'est pas encore présent.

8.4. Améliorations proposées

Pour améliorer le projet, plusieurs pistes peuvent être envisagées :

1. utiliser `GridSearchCV`, `RandomizedSearchCV` ou `Optuna` pour optimiser les hyperparamètres ;
2. tester `class_weight` ou `SMOTE` pour traiter le déséquilibre des classes ;
3. ajouter une validation croisée ;
4. analyser l'importance des variables ;
5. ajouter une matrice de confusion et des courbes ROC/PR ;
6. documenter l'API avec `Swagger` ou `OpenAPI` ;
7. conteneuriser l'application avec `Docker` ;
8. mettre en place un suivi du data drift.

9. Conformité avec les exigences de l’atelier

Le projet répond aux principales exigences de l’atelier Machine Learning. Il comprend les éléments attendus : structure de dossiers, notebook d’exploration, scripts de preprocessing, modèles de classification, clustering, régression, visualisations, sauvegarde des modèles et application Flask.

Exigence	Fichier / module	Statut
Structure de projet claire	Dépôt GitHub	Conforme
Fichier requirements.txt	requirements.txt	Présent
Notebook d’exploration	notebooks/	Présent
Prétraitement complet	preprocessing.py	Implémenté
Parsing des dates	preprocessing.py	Implémenté
Feature engineering	preprocessing.py	Implémenté
Correction des valeurs aberrantes	preprocessing.py	Implémenté
Imputation KNN	preprocessing.py	Implémenté
Standardisation	preprocessing.py	Implémenté
Encodage ordinal et one-hot	preprocessing.py	Implémenté
Target encoding	preprocessing.py	Implémenté
ACP	pca_transform.py	Implémenté
Classification Random Forest	train_classification.py	Implémenté
Régression logistique	train_model.py	Implémenté
Clustering KMeans	train_clustering.py	Implémenté
Régression	train_regression.py	Implémenté
Déploiement Flask	app.py	Implémenté
Module de prédiction	predict.py	Implémenté

TABLE 6: Grille de conformité avec les exigences de l’atelier

10. Conclusion

Ce projet constitue une application complète du machine learning à un problème réel de relation client. Il permet de prédire le churn, de segmenter les clients et de déployer un modèle dans une application web. Le travail couvre l'ensemble du cycle de vie d'un projet ML : compréhension métier, préparation des données, modélisation, évaluation et déploiement.

L'approche retenue est cohérente avec les objectifs de l'atelier. Le Random Forest constitue un choix pertinent pour la prédiction du churn, tandis que l'ACP et KMeans apportent une dimension exploratoire et analytique utile pour la segmentation client.

Les principales perspectives d'amélioration concernent l'optimisation des hyperparamètres, la gestion avancée du déséquilibre des classes, l'amélioration de l'évaluation expérimentale et l'industrialisation du déploiement. Avec ces améliorations, le projet pourrait évoluer vers une solution plus robuste, plus fiable et plus proche d'un environnement de production.